MANAJEMEN KEUANGAN

NAMA : M.Danuarta Wiraguna

NIM : F1D02310124

KELAS : C

1. Deskripsi Aplikasi dan Tujuan Projoect

a. Deskripsi Aplikasi

Aplikasi "Manajemen Keuangan" adalah sebuah perangkat lunak berbasis desktop yang dirancang untuk membantu pengguna dalam mencatat, mengelola, dan memantau transaksi keuangan sehari-hari, baik berupa pemasukan maupun pengeluaran. Dikembangkan menggunakan bahasa pemrograman Python dan kerangka kerja antarmuka grafis (GUI) PySide6, aplikasi ini menawarkan pengalaman pengguna yang interaktif, sederhana, dan responsif. Untuk memastikan data pengguna tidak hilang setelah aplikasi ditutup (persistensi data), sistem ini diintegrasikan dengan Relational Database Management System (RDBMS) ringan, yaitu SQLite, sehingga seluruh data transaksi dapat tersimpan dengan aman dan dapat diakses kembali kapan saja.

b. Tujuan Proyek

Pengembangan proyek ini memiliki beberapa tujuan utama:

1) **Implementasi Arsitektur Perangkat Lunak:** Menerapkan prinsip *Separation of Concerns* (SoC) untuk menghasilkan kode program yang terstruktur, mudah dipelihara (*maintainable*), dan mudah dikembangkan (*scalable*).
2) **Efisiensi Pengelolaan Keuangan:** Menyediakan alat bantu yang sederhana dan praktis bagi pengguna untuk mencatat serta mengelola transaksi pemasukan dan pengeluaran secara terstruktur dan sistematis.
3) **Penerapan Komponen GUI Lanjutan:** Mengimplementasikan konsep *signals and slots*, manajemen tata letak (*layout management*), penggunaan dialog interaktif, serta penerapan stylesheet eksternal (*QSS*) dalam pengembangan antarmuka berbasis PySide6.
4) **Penyimpanan Data Secara Persisten:** Mengintegrasikan database SQLite sebagai media penyimpanan data agar informasi transaksi dapat disimpan secara permanen dan dapat diakses kembali kapan saja.

2. Struktur Project dan Penerapan Separation of Concerns (SoC)

Aplikasi ini dibangun dengan mematuhi prinsip Separation of Concerns (SoC), yaitu pemisahan program ke dalam beberapa modul berdasarkan tanggung jawab masing-masing. Pendekatan ini bertujuan untuk menghindari terjadinya *Spaghetti Code* (kode yang tidak terstruktur dan sulit dipahami), serta meningkatkan keterbacaan, kemudahan perawatan, dan pengembangan aplikasi di masa mendatang.

Struktur direktori aplikasi disusun sebagai berikut:

1) main.py

Bertindak sebagai titik awal eksekusi program. Modul ini bertanggung jawab untuk menginisialisasi aplikasi menggunakan *QApplication*, memanggil fungsi pembuatan tabel pada database, memuat file stylesheet eksternal (QSS), serta menampilkan jendela utama aplikasi. Modul ini hanya berfokus pada pengaturan alur awal program dan tidak menangani secara langsung logika database maupun desain antarmuka.

2) ui_main.py

Modul yang mengelola antarmuka utama aplikasi. Bertanggung jawab dalam pembuatan dan pengaturan komponen seperti tabel data (*QTableWidget*), tombol aksi (tambah, edit, hapus), label informasi, serta menu bar. Selain itu, modul ini juga mengatur tata letak (*layout management*) agar tampilan aplikasi lebih terstruktur dan mudah digunakan.

3) ui_dialog.py

Modul khusus untuk menangani dialog (*QDialog*) untuk proses input dan pengeditan data transaksi. Pemisahan modul ini bertujuan agar logika form tidak membebani kelas utama (*Main Window*), sehingga struktur program tetap modular dan lebih mudah dipahami.

4) database.py

Modul yang bertanggung jawab penuh dalam pengelolaan database menggunakan SQLite. Semua operasi seperti pembuatan tabel (*CREATE TABLE*), penambahan data (*INSERT*), pengambilan data (*SELECT*), pembaruan data (*UPDATE*), dan penghapusan data (*DELETE*) ditangani di dalam modul ini. Dengan demikian, bagian antarmuka tidak perlu berinteraksi langsung dengan perintah SQL.

5) style.qss

Modul yang berisi pengaturan tampilan visual aplikasi, seperti warna, font, ukuran, padding, dan elemen desain lainnya. Dengan memisahkan stylesheet ke dalam file eksternal, perubahan tampilan dapat dilakukan dengan mudah tanpa harus mengubah kode utama aplikasi.

3. Desain Database

Aplikasi menggunakan SQLite sebagai mesin basis data karena sifatnya yang *serverless* dan terintegrasi langsung dalam file lokal (`penjadwal.db`). Tabel tunggal dirancang untuk menyimpan entitas jadwal.

Nama Kolom	Tipe Data	Constraints	Deskripsi
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Penanda unik (*identifier*) untuk setiap aktivitas. Dibuat otomatis oleh sistem.

nama	TEXT	NOT NULL	Nama atau judul transaksi yang diinput oleh pengguna (misalnya beli makan, dan gaji).
tanggal	TEXT	NOT NULL	Tanggal terjadinya transaksi(Format: YYYY-MM-DD).
jenis	TEXT	NOT NULL	Jenis transaksi, yaitu “Pemasukan” atau “Pengeluaran”.
jumlah	TEXT	NOT NULL	Nominal uang dari transaksi yang dilakukan.
keterangan	TEXT	-	Tingkat Informasi tambahan atau deskripsi terkait transaksi (opsional).

4. Fitur Utama dan Cara Kerjanya

a. Form Input Dinamis (Dialog Terpisah)

1) **Cara Kerja:** Ketika pengguna mengklik tombol "Tambah" pada jendela utama, aplikasi akan memicu sinyal yang membuka jendela dialog (*FormDialog*). Form ini berisi beberapa komponen GUI seperti *QLineEdit* (untuk nama, tanggal, jumlah, dan keterangan) serta *QComboBox* (untuk memilih jenis transaksi: Pemasukan atau Pengeluaran). Data yang diinput oleh pengguna kemudian dikumpulkan dan dikirim kembali ke *controller* untuk diproses.
2) Validasi input juga diterapkan pada tahap ini; sistem akan menampilkan *QMessageBox* sebagai peringatan apabila terdapat field penting yang kosong atau jika nilai jumlah bukan berupa angka sebelum data disimpan ke database.

b. Manajemen Data (Read & Delete)

1) **Cara Kerja (Create & Read):** Setelah pengguna menambahkan data, fungsi `tambah()` pada `main.py` akan memanggil fungsi `INSERT` pada `database.py`. Selanjutnya, fungsi `load_data()` akan mengambil seluruh data dari database menggunakan perintah `SELECT` dan menampilkannya ke dalam *QTableWidget* secara dinamis. Data ditampilkan dalam bentuk tabel agar mudah dibaca dan dikelola.
2) **Cara Kerja (Update & Delete):** Untuk proses edit, pengguna harus memilih salah satu baris pada tabel, kemudian menekan tombol "Edit". Data lama akan dimuat kembali ke dalam dialog untuk diperbarui, lalu disimpan menggunakan perintah UPDATE. Sedangkan untuk penghapusan, sistem akan mengambil id dari baris yang dipilih dan menampilkan

`QMessageBox` sebagai konfirmasi sebelum menjalankan perintah DELETE. Setelah proses selesai, tabel akan diperbarui secara otomatis (refresh).

c. Integrasi Identitas Statis & Bilah Menu

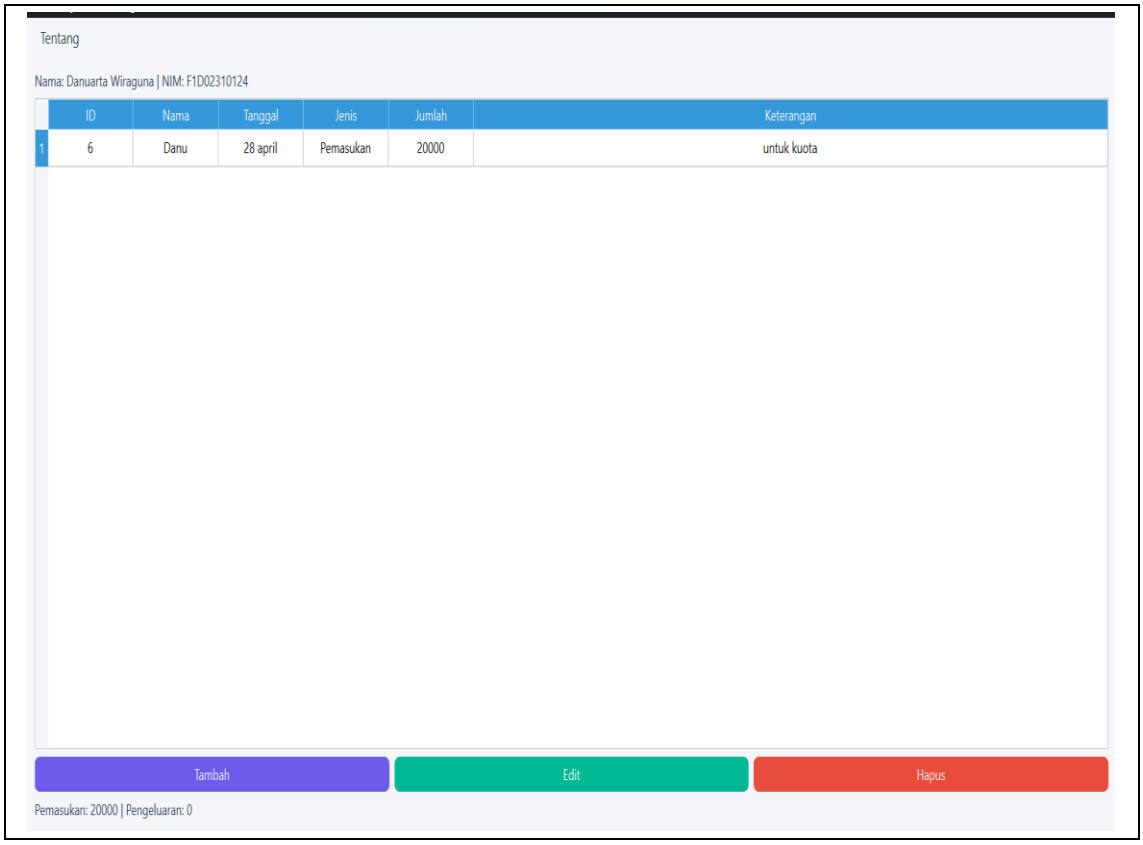
Sesuai dengan ketentuan, aplikasi menampilkan identitas mahasiswa berupa nama dan NIM yang ditampilkan menggunakan komponen `QLabel` pada bagian atas antarmuka. Selain itu, terdapat `QMenuBar` yang berisi menu "Tentang" yang ketika dipilih akan menampilkan informasi aplikasi melalui `QMessageBox`, seperti nama aplikasi, identitas pengembang, serta deskripsi singkat program.

d. Kustomisasi Visual (Theming)

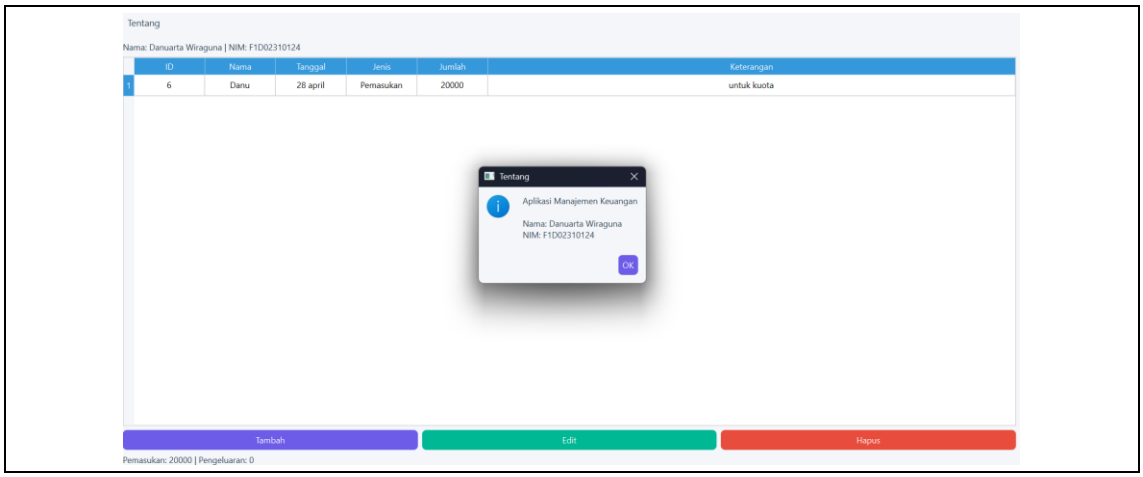
Saat Saat aplikasi dijalankan melalui `main.py`, sistem akan membaca file `style.qss` dan menerapkannya ke seluruh komponen GUI menggunakan fungsi `setStyleSheet()`. Dengan pendekatan ini, tampilan aplikasi menjadi lebih menarik melalui penggunaan warna, efek hover pada tombol, tampilan tabel yang lebih modern, serta penyesuaian elemen antarmuka lainnya tanpa harus mengubah kode utama Python.

5. Screenshot antarmuka

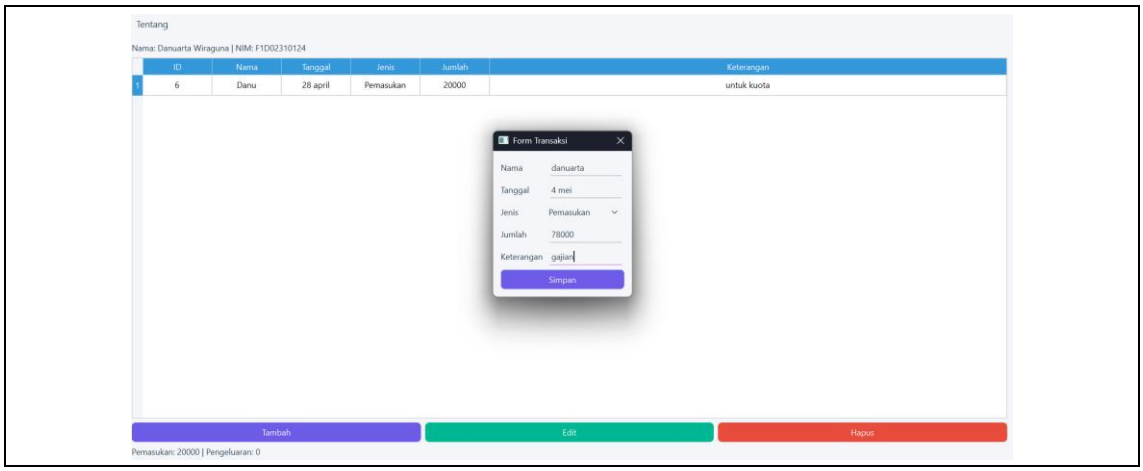
a. Tampilan awal aplikasi



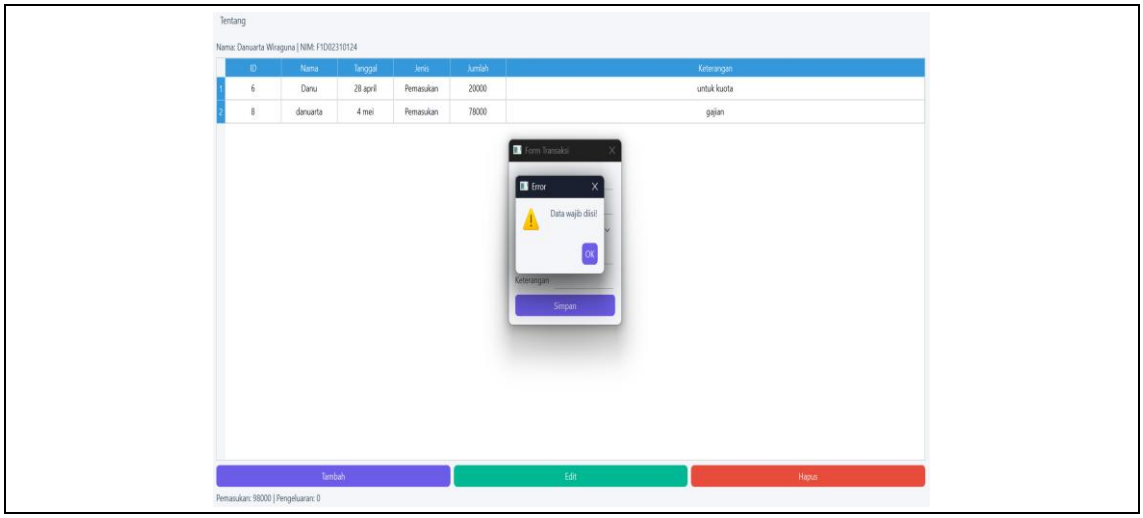
b. Tentang aplikasi



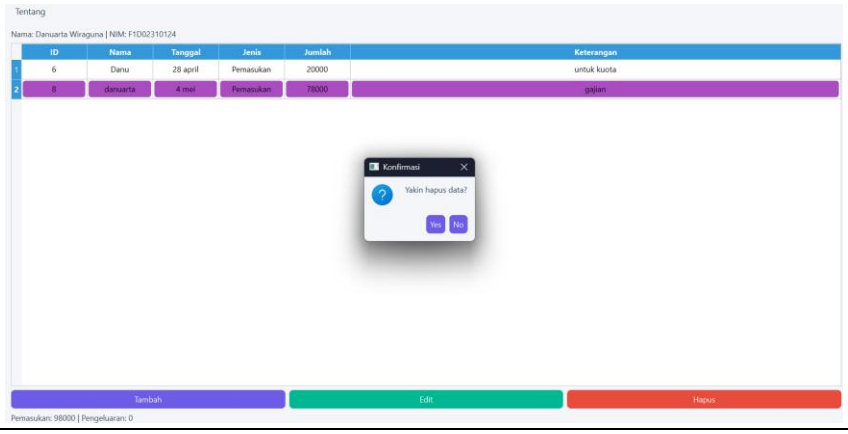
c. Tambah transaksi



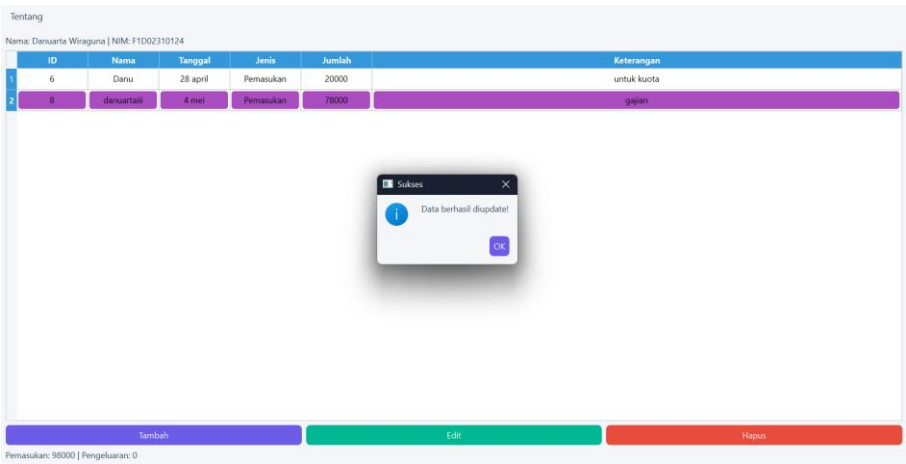
d. Peringatan ketika tidak mengisi data



e. Konfirmasi ketika ingin menghapus data



f. Konfirmasi Ketika update data



6. Link YouTube

https://youtu.be/zDfqlwkbqwU?si=ToM9dTzusMVQptkj

7. Link GitHub

https://github.com/B4R0N9/pv26-miniproject-manajemen-keuangan-F1D02310124.git